Two-product checkout offer: evidence, model, and economics

Reproducible analysis of the supplied transaction CSV | 17 October 2026

Executive decision

A customer selects one product; the prototype offers exactly two complementary products for 15 seconds. Choosing either add-on in time applies a 15% discount to that add-on only. The historical data can support which products to test and the unit economics. It cannot establish that this promotion increases revenue or profit.

Recommendation: use the working local demo and run a randomized checkout pilot. Measure total basket gross profit and revenue per eligible checkout against a no-offer control. Do not describe the scenario figures in this report as a forecast.

Evidence class	What is known
Observed	33,887 completed transactions, $305,925.14 recorded revenue.
Backtested	Historical top-two basket prediction on later months; not offer response.
Scenario	Financial effects under explicit assumed incremental and baseline-buyer rates.
Unknown	Actual offer exposure, incremental conversion, substitution, labor, inventory, and net profit.

Data source: stories_transactions.csv only. Transaction IDs begin with SYN-, a provenance signal that should be clarified before operational deployment. All final figures are regenerated by the repository code.

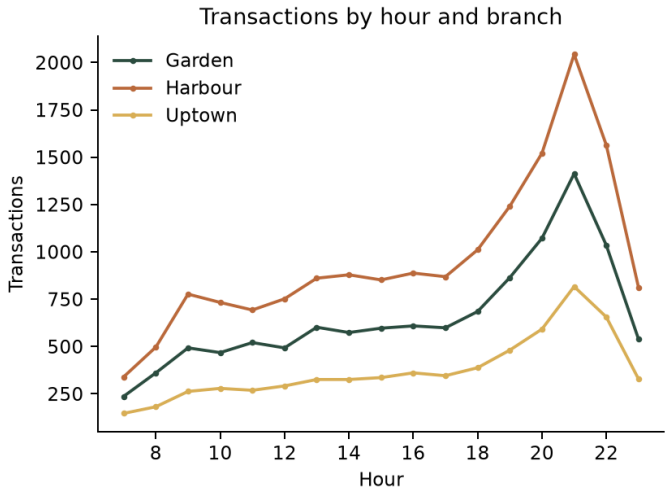
1. Data audit and scope

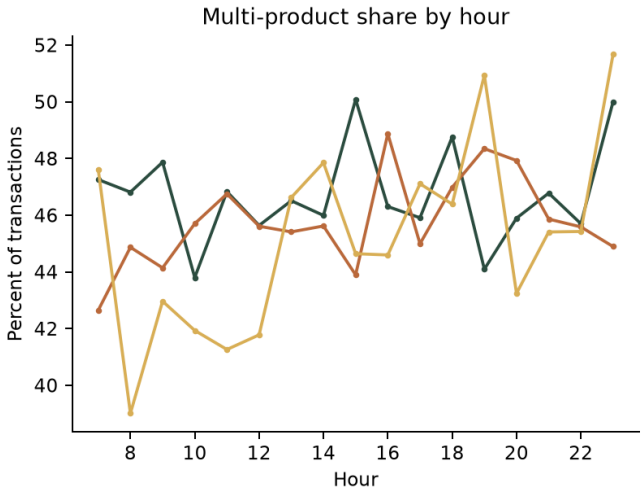
Measure	Result
Product lines	51,177
Transactions	33,887
Period	2026-01-01 to 2026-06-30
Branches / products / categories	3 / 45 / 9
Recorded revenue	$305,925.14
Recorded gross profit	$179,430.49
Missing customer ID rows	18,691
Transactions with a discounted line	3,247

Transaction ID defines the basket. Pair counts use distinct product presence, while financial totals preserve quantity and recorded line revenue/gross profit. Date, time, branch, order type, and customer ID are checked for within-transaction consistency. Source amounts reconcile within two cents per line; cent differences may reflect rounding. Customer ID is omitted from the recommender because of missingness and lack of a verified personalization benefit.

Offline pseudo-checkout: the file does not contain entry sequence. Each multi-product basket is evaluated from every possible seed item with total weight one per transaction. This estimates retrospective ranking quality, not an actual point-of-sale impression or response.

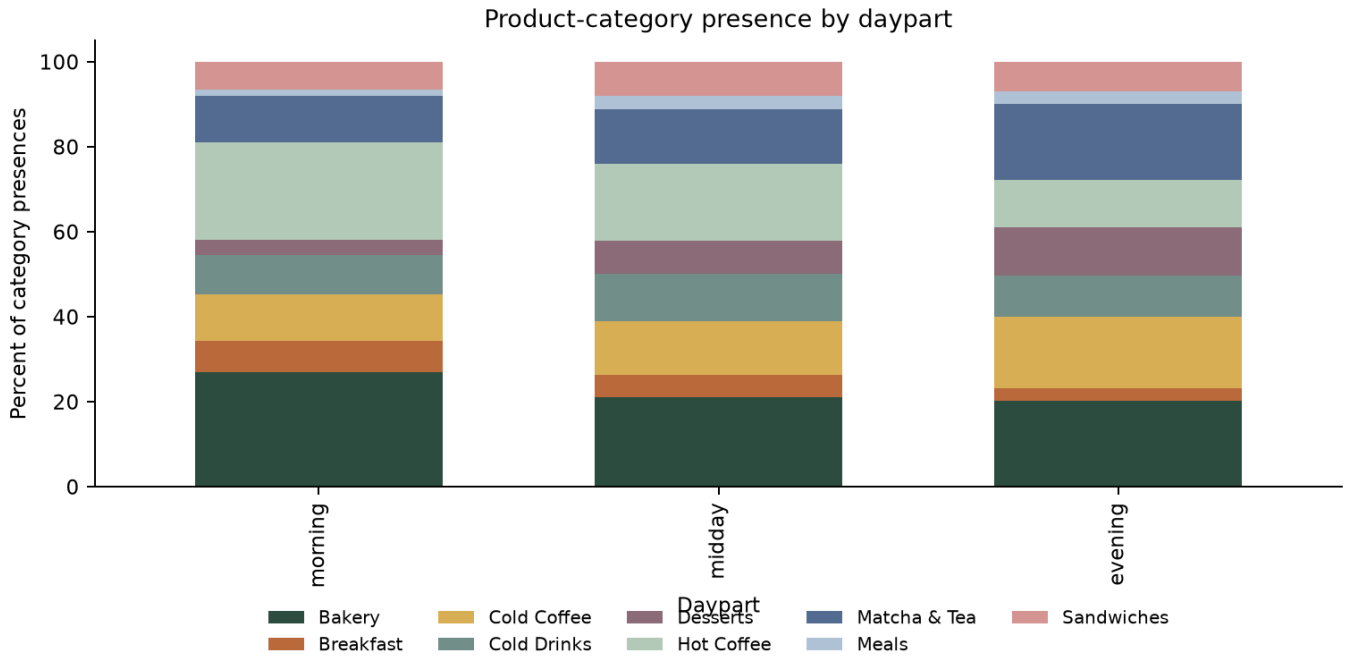
2. Time and branch





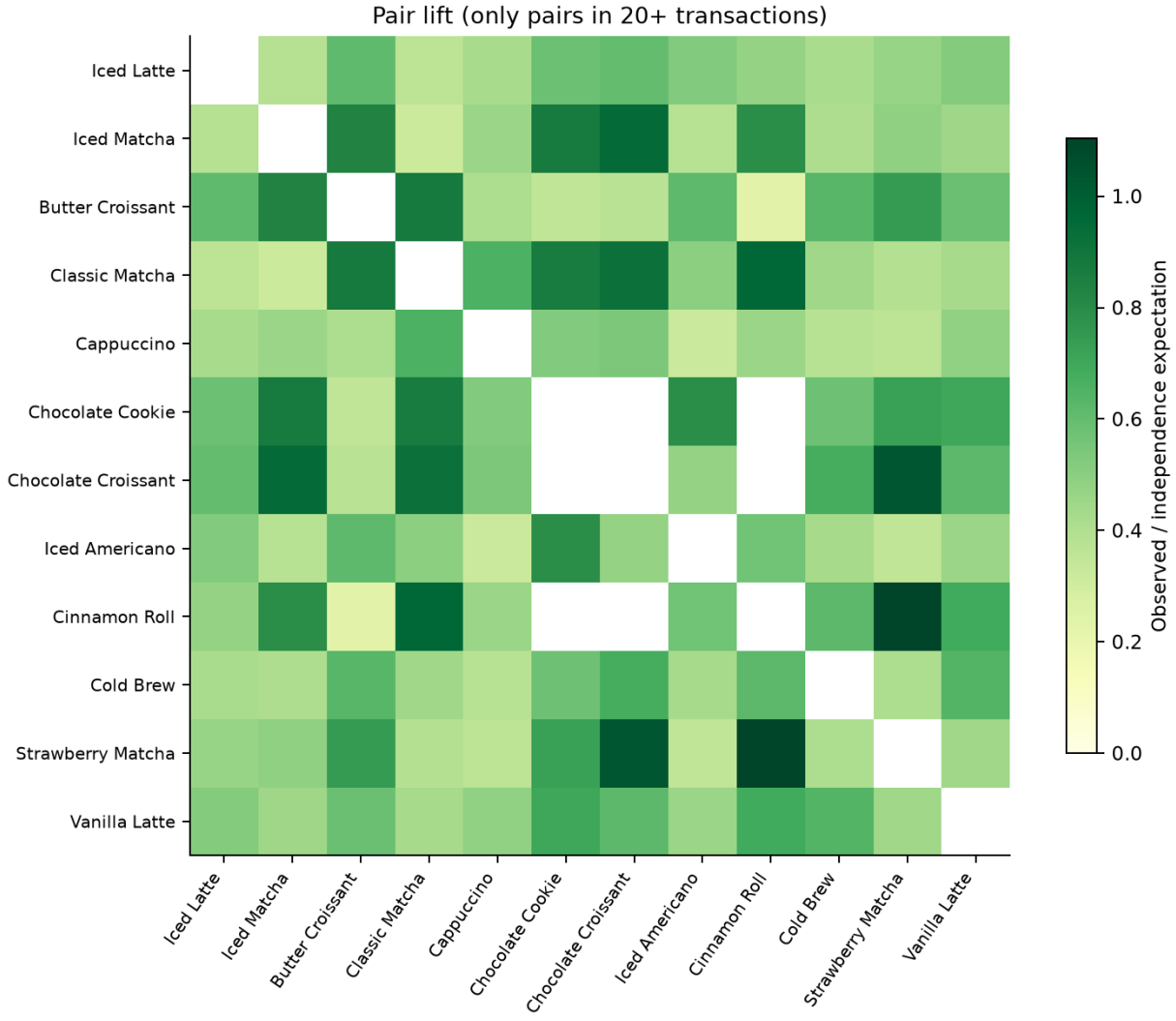
Counts use transactions per branch-hour. Multi-product share divides multi-product transactions by all transactions in each branch-hour. Traffic is not a measure of queue length or capacity.

3. Category mix by daypart



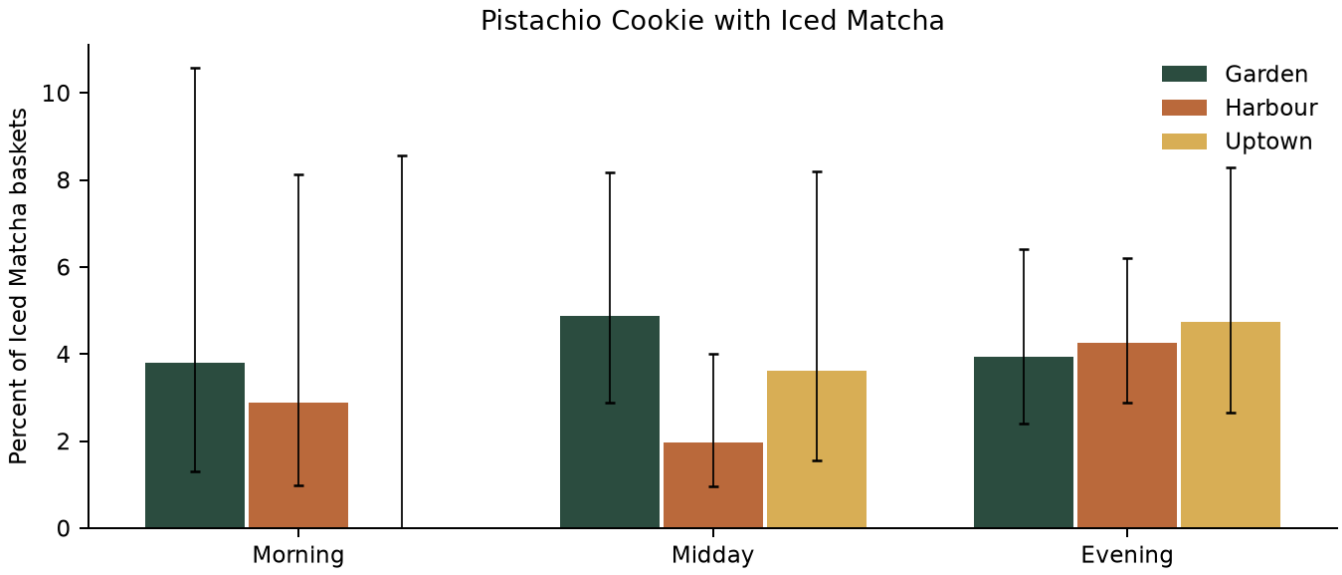
Each transaction/category presence is counted once. Shares are within daypart and describe product mix, not a probability that a recommendation will convert.

4. Product-pair association



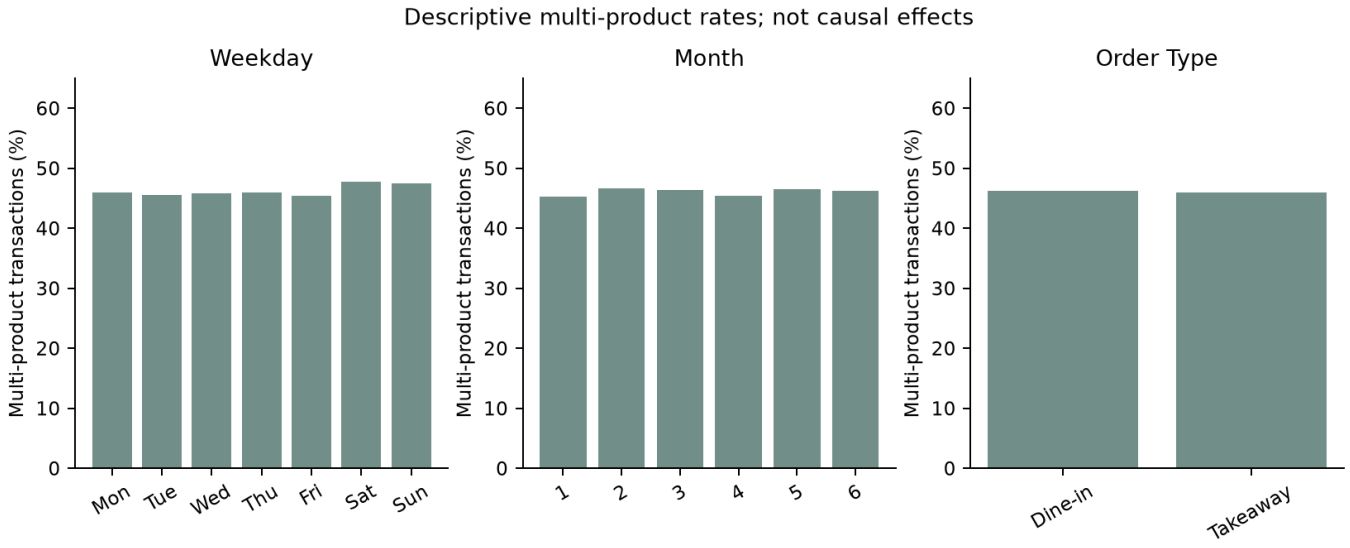
Lift compares observed pair frequency with the expectation under unconditioned independence. Only pairs with at least 20 transactions are shown; lift remains associational and can reflect context and basket size.

5. Example pairing



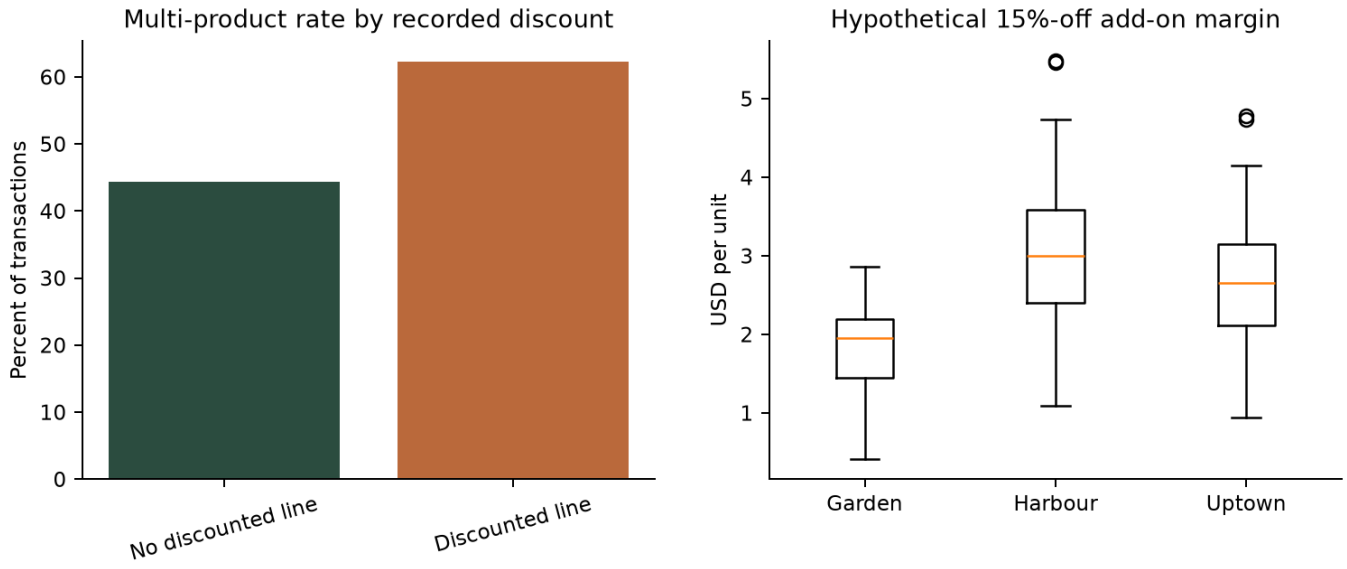
Iced Matcha and Pistachio Cookie occur together in 82 completed transactions. Bars divide pair baskets by seed-containing baskets in each branch and daypart; detailed cell counts are in outputs/results.json.

6. Other factors



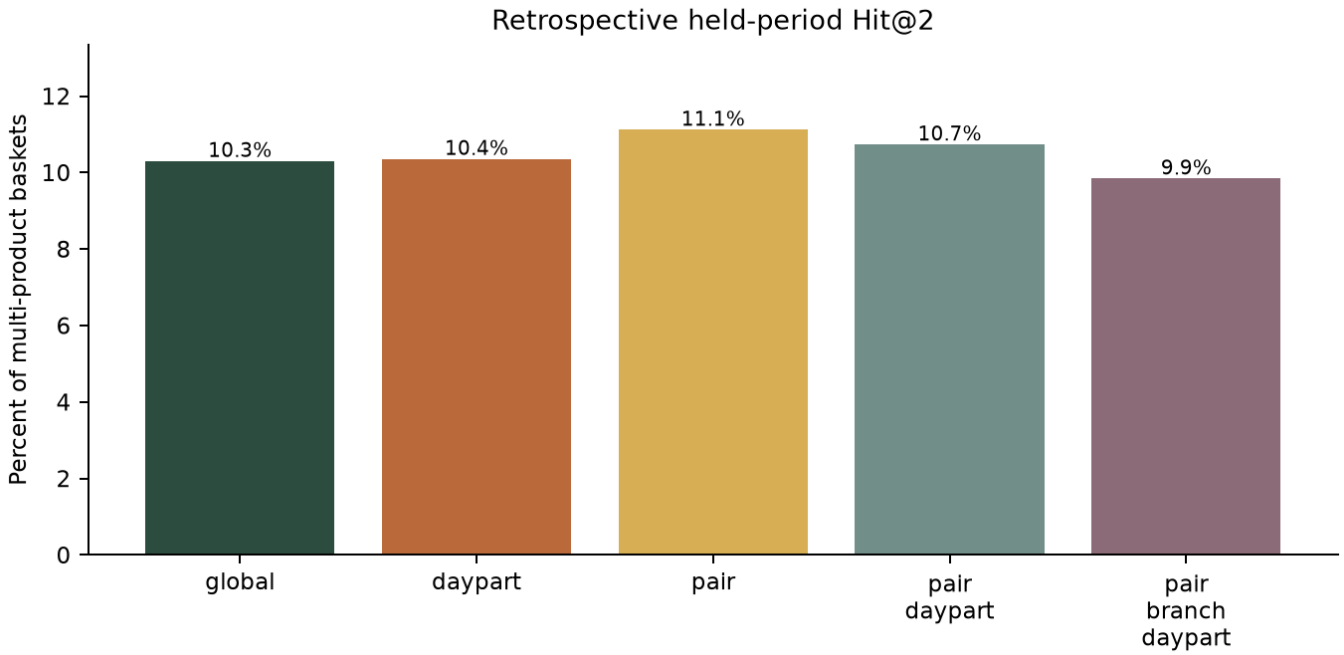
Weekday, month, and order type are plotted even when differences are small. Rates use all completed transactions in each group. Effects can be confounded by branch, product mix, and traffic.

7. Discount and margin



The left panel is a descriptive split by whether any line was discounted; it is not a causal discount estimate. The right panel uses branch-product median price and cost to calculate a hypothetical 15%-off unit margin.

8. Retrospective model comparison



Models fit January-March and evaluate April-June. Best plotted Hit@2 is pair; that choice is exploratory because later months were already discussed in prior project context.

9. Model and validation

The live recommender is a smoothed product-pair co-purchase table. It records pair counts and seed-product exposures, with global popularity as a prior for sparse pairs. The benchmark also tests daypart and branch conditioning with additional shrinkage. The live rule returns two distinct menu products with positive hypothetical discounted unit gross profit. A completed local demo order updates counts once using its event ID; a displayed suggestion does not.

Rule	Hit@1	Hit@2 (95% CI)	MRR@2
global	5.3%	10.3% (9.8%-10.8%)	0.078
daypart	5.5%	10.4% (9.9%-10.8%)	0.079
pair	5.9%	11.1% (10.6%-11.6%)	0.085
pair_daypart	5.8%	10.7% (10.2%-11.2%)	0.083
pair_branch_daypart	5.2%	9.9% (9.4%-10.3%)	0.075

Training transactions: 14,792. Evaluation transactions: 19,095; the metric denominator is multi-product transactions in that period with a pseudo-seed. Hit@2 means at least one actual other product is in the two suggestions. It does not measure add-on conversion or causal sales uplift.

The selected pair rule has 33.7% category-level Hit@2 and 100% candidate coverage across evaluated seeds. The reported 95% intervals use transaction-level standard errors; they describe retrospective ranking uncertainty, not uncertainty in business uplift.

Month	Multi-product test baskets	Hit@2
2026-04	2,615	11.5%
2026-05	2,994	10.9%
2026-06	3,192	10.5%

A neural network is not used: the dataset is small, there is no intervention-response label, and a count-based model is directly updatable and auditable. Adding daypart and branch reduced Hit@2 in this retrospective evaluation, so the app uses the simpler product-pair policy. Time remains available for future testing when new intervention data arrive.

10. Financial sensitivity - example, not forecast

Example: Harbour, first item Iced Matcha, offered add-on Pistachio Cookie. Median add-on price $3.62; median recorded unit cost $1.18. At 15% off, the rounded add-on selling price is $3.08 and unit gross profit before extra operating costs is $1.90.

The CSV contains 1,048 Harbour baskets with the seed and 375 completed seed-only baskets. Neither count is a known number of live offer impressions.

Per eligible offer, under the stated simplifying assumptions: incremental revenue = newly added rate x discounted price - rate of existing buyers receiving a discount x concession - substituted revenue. Incremental gross profit replaces price with discounted unit gross profit, then subtracts discount leakage, substitution, and added variable operating cost. All response rates are unknown until tested.

Assumed new add-on	Existing buyers discounted	Revenue / offer	Gross profit / offer
3%	0%	$0.092	$0.057
3%	5%	$0.065	$0.030
3%	10%	$0.038	$0.003
5%	0%	$0.154	$0.095
5%	5%	$0.127	$0.068
5%	10%	$0.100	$0.041
10%	0%	$0.308	$0.190
10%	5%	$0.281	$0.163
10%	10%	$0.254	$0.136

At an assumed 5% existing-buyer discount leakage and zero substitution/extra cost, new add-on conversion would need to exceed 1.4% for positive incremental gross profit in this simplified example. Larger operating or substitution costs raise this threshold.

11. Portfolio-scale sensitivity

There are 18,256 completed one-product transactions. Using them as a historical sizing proxy, the final product-pair model generates two offers for each basket. An equal mix of the two offers has mean discounted selling price $4.52, mean discounted unit gross profit $2.47, and mean discount concession $0.80. These item amounts are computed from the CSV; the 50/50 choice mix and response rates are assumptions. One-product completed receipts are not known live offer impressions.

The following six-month numbers multiply per-offer scenarios by that sizing proxy. They are **not forecasts**. Net scenarios additionally assume $0.25 per newly added item, $0.02 per offer, $25 monthly fixed expense, and $500 one-time development expense. No such operating cost is recorded in the CSV; replace each assumption with a quote or pilot observation before a rollout decision.

New add-on	Existing buyer discount	Extra revenue	Extra gross profit	Net after assumed costs
3%	0%	$2,474	$1,353	$201
3%	5%	$1,747	$626	$-526
3%	10%	$1,020	$-101	$-1,253
5%	0%	$4,123	$2,256	$1,012
5%	5%	$3,396	$1,529	$285
5%	10%	$2,669	$801	$-442
10%	0%	$8,247	$4,512	$3,040
10%	5%	$7,520	$3,784	$2,313
10%	10%	$6,793	$3,057	$1,586

These calculations assume no substitution and no effect on first-item sales. Either could make the outcome worse. Conversely, repeat visits or a different offer mix could change it. The randomized pilot must measure full baskets, not only recommended-item redemptions.

12. Marketing, operations, and pilot

Customer presentation: after the first item, show two relevant add-ons, each original and discounted price, and a plain 15-second timer. Discount only a selected add-on. Monitor whether urgency increases abandonment or dissatisfaction. Branch and daypart can be used for targeting only when retrospective evidence supports them.

Cost	CSV evidence	Needed before rollout
Discount concession	Listed price and 15% rule	Actual redemptions and baseline buyers
Product cost	Recorded unit cost by branch	Confirm cost definition
Development / hosting	None	Time log and supplier quote
Staff / prep / packaging	None	Observed incremental minutes and costs
Stockouts / substitution	None	Inventory and full-basket outcomes

Minimum randomized pilot: assign eligible checkouts before the offer is displayed to either ordinary checkout or the exact two-item, 15-second, 15%-off experience. Keep assignment stable on refresh and log both shown items, expiry, response, completed basket, revenue, gross profit, and order completion. Analyze all assigned checkouts, including refusals/timeouts. Primary endpoint is total basket gross profit per eligible checkout with a confidence interval; revenue per eligible checkout is a co-primary business view or key secondary metric. Obtain pilot variance before final sample sizing.

If volume permits, add an otherwise identical no-discount arm to separate suggestion effects from the discount. Measure baseline-buyer discount leakage and substitution across the whole basket. Scale only when incremental contribution exceeds operating costs without unacceptable customer or service effects.

Reproduction: run `python -m src.pipeline` from the repository root, then `streamlit run app.py`. The CSV remains the only source of historical business figures; outputs/results.json contains the complete computed tables and cell counts.